OrthoSense React Native Mobile App Guide

A gradual plan for building a secure Android-first companion and BLE gateway app for the rehabilitation-monitoring prototype.

Purpose. This guide shows how to build a mobile app that can later connect to the ESP32-S3 wearable, display honestly labelled device and sensor status, and use confirmed backend services. Start with a local, synthetic app and add BLE and cloud features only after each foundation works.

> ***Important boundary.*** *OrthoSense is a university prototype, not a clinically validated medical device. The app must not diagnose, prescribe treatment, recommend rehabilitation limits, or present mock data as live telemetry.*

Recommended first architecture

Layer	Choice	Reason
Mobile framework	React Native + TypeScript	One typed codebase for Android and later iOS.
Starter workflow	Expo development build	Fast initial setup; needed after adding native BLE support.
Navigation	Expo Router or React Navigation	Typed screen flow and safe mobile navigation.
Device link	BLE only after protocol is defined	The current documents describe BLE as planned, not confirmed.
Cloud link	HTTPS to confirmed backend	Do not invent endpoints, fields, or authentication rules.

Decision: Begin Android-first because you are developing on Windows. Add iOS when you have access to a Mac and an Apple developer workflow. Keep the current Next.js app as the clinician web experience; the mobile app is a separate companion/gateway codebase.

1 What the mobile app should do

Build the first release around two clear user journeys. Do not try to build every planned web route on day one.

Patient companion journey

- Sign in, then view only the patient’s permitted profile, session history, own device status, and clearly labelled measurement freshness.
- Pair with a confirmed wearable device only after the device identity, BLE services, and authorization model are agreed.
- Show local transfer state: scanning, connecting, connected, reconnecting, offline, failed, and last successful data time.
- Upload queued telemetry only through the confirmed gateway/backend contract. A queued upload is not proof that the cloud received it.

Clinician companion journey

- Review permitted assigned-patient summaries, devices, sessions, and alerts.
- Acknowledge an alert only after the backend confirms authorization and success.
- Show alert status separately from device status. Acknowledgement does not resolve a sensor/device condition.

The data flow you are working toward

Wearable ESP32-S3 -> Bluetooth Low Energy -> React Native mobile app -> authenticated HTTPS -> backend -> web and mobile views. The documents currently describe BLE telemetry and a mobile gateway as planned capabilities, so treat this as an architecture target until the edge and backend teams confirm the exact protocol.

App state	What the user sees	What it must not claim
Demo	Synthetic demonstration data	That it came from a wearable or patient
BLE connected	Connected to a named device; last packet time	That measurements are calibrated or uploaded
BLE stale	Last packet age and retry option	That current readings are live
Cloud queued	Pending upload count	That the backend accepted it
Cloud confirmed	Server-confirmed sync time	That clinical review occurred

2 Set up the React Native project

Use TypeScript from the first commit. For a project that will eventually use BLE, start with Expo but plan on an Expo development build rather than relying on Expo Go after native modules are introduced. Expo documents that adding native-code libraries requires rebuilding the development client.

Install on Windows

- Install a current Node LTS release, Android Studio, Android SDK, and an Android emulator or USB-debuggable Android phone.
- Install the Expo tooling and create the app in the existing D:\3yp\mobile_app folder only after checking whether it already contains user work.
- Use Android Studio for the emulator. Test BLE later on a physical phone: emulators cannot prove Bluetooth behavior.

Create a minimal typed app

Example commands to run after confirming the target folder is safe to initialize:

```
npx create-expo-app@latest mobile_app --template tabs
cd mobile_app
npx expo start
```

The template choice is a starter only. Keep its navigation until you understand it, then rename the tabs to match your app. Do not install every possible mobile library at once.

Baseline dependencies

Need	Recommended approach	When to add
Navigation	Expo Router from the starter or React Navigation	At project creation
Secure tokens	expo-secure-store	When real sign-in exists
Network state	NetInfo or platform-supported equivalent	Before offline upload
Persistent queue	SQLite or a reviewed local store	Before BLE/cloud relay
BLE	A maintained native BLE library	Only after GATT protocol is written
Charts	A reviewed chart solution plus text/table alternative	After real measurement definitions exist

React Navigation provides typed routing and navigation patterns. Its current documentation lists React Native, Expo, and TypeScript compatibility requirements; install package versions through the Expo installer so they match your selected Expo SDK.

3 Design the codebase to scale

Separate screens from business logic, BLE transport, HTTP services, storage, and domain models. This keeps the app testable and prevents a UI component from becoming the place where device protocol and cloud authorization decisions are hidden.

Suggested folder layout

```
mobile_app/
  app/                        # routes/screens
  src/
    components/               # accessible shared UI
    features/
      auth/
      devices/
      sessions/
      alerts/
      gateway/
    domain/                   # types and pure rules
    services/                 # backend interfaces and adapters
    ble/                      # scan, connection, protocol adapters
    storage/                  # token store and offline queue
    config/                   # non-secret environment configuration
    test/                     # fixtures and test helpers
  assets/
  app.json
  package.json
```

Keep the boundary typed

Use explicit status unions rather than display strings. The following aligns with the planning documents but remains a client-side model until the backend contract is approved.

```
export type ConnectionStatus =
  | "connected"
  | "disconnected"
  | "stale"
  | "unknown";

export type MeasurementQuality =
  | "valid"
  | "stale"
  | "missing"
  | "invalid"
  | "estimated";

export interface Measurement {
  id: string;
  deviceId: string;
  timestamp: string;
  type: "imu" | "fsr" | "temperature" | "vibration";
  value?: number;
  rawValue?: number;
  unit?: string;
  quality: MeasurementQuality;
  source: "demo" | "edge" | "gateway" | "backend";
}
```

A raw FSR reading must remain labelled as raw ADC counts until hardware calibration supports a force estimate. A motion estimate, an ML result, and raw IMU axes are distinct data products and must not share a misleading label.

4 Build the UI in the right order

Use hard-coded content only in a clearly labelled demo adapter. All later screens should read through a service interface, so replacing demo data with a confirmed backend does not require rewriting every screen.

Milestone	Screens	Acceptance checks
M1 App shell	Sign-in placeholder, home, settings, empty/error/offline states	App builds; tab order, labels, contrast, and screen-reader names work.
M2 Demo device	Device list, device details, sensor health card	Every value says Demo; unknown/missing/stale are different.
M3 Sessions	Session list and detail; textual data view	No fabricated gaps or zero replacement; units and time shown.
M4 Alerts	Alert list/detail and permitted acknowledgement UI	Pending/failed/success action states are distinct.
M5 BLE gateway	Scan, device selection, connection, relay status	Physical Android testing proves the chosen protocol path.
M6 Backend	Sign-in, authorized data, queued upload state	Contract validation, error handling, and permission checks pass.

Accessibility rules

- Use a visible label for every icon button and accessibilityLabel for assistive technology.
- Do not communicate connection or alert state by color alone. Include text and, where useful, an icon.
- Use readable touch targets, dynamic text support, logical focus order, and error text next to the control that caused it.
- For charts, show units, time range, status, and a textual/table alternative. Never interpolate over missing samples without saying so.

Initial home screen

Show a compact summary: selected role, device status, last successful data time, queue/sync state, recent permitted sessions, and any alerts. Do not show a reassuring “Online” state during loading, failure, stale data, or unknown state.

5 Add BLE only after the device protocol exists

BLE is not simply “connect and receive a value.” The ESP32 firmware and mobile app need one written protocol. Implement a synthetic BLE adapter or mock service first so the UI flow can be tested before physical hardware is ready.

Write this protocol contract with the edge team

Decision	Questions to answer
Device identity	How is a wearable uniquely identified? What is shown to the user?
GATT services	Which service UUIDs, characteristic UUIDs, directions, encoding, and maximum packet sizes?
Telemetry	Which readings, units, timestamps, sequence IDs, data quality, and source version?
Connection	Pairing method, reconnect behavior, timeout, and duplicate-device rules?
Commands	Which commands are allowed? Who authorizes them? How does the device acknowledge?
Security	Does BLE pairing suffice, or is application-level device authentication required?

BLE implementation sequence

- Request Android Bluetooth permissions with clear user-facing reasons. Treat refusal as a state, not an error to bypass.
- Scan only after user action. List only allowed device information; stop scanning when it is no longer needed.
- Connect to the selected device, discover services, and verify that the expected characteristics exist.
- Subscribe to notifications, validate packet length/version/checksum if defined, and decode to a typed record.
- Update last-success time only after a valid packet is decoded. An open BLE connection alone does not prove fresh telemetry.
- Use sequence IDs to detect duplicates and gaps. Queue valid records locally before upload; mark them as pending until the server confirms receipt.
- Disconnect intentionally when the user ends the session or the app is shutting down. Reconnect with bounded backoff, never a tight retry loop.

Reality check: *BLE features require a physical device for meaningful verification. A simulator can demonstrate UI states but cannot verify Android permissions, radio behavior, pairing, packet loss, or the ESP32 GATT implementation.*

6 Backend integration and offline relay

The available OrthoSense documents provide proposed models and service boundaries, not confirmed backend URLs. Build service interfaces first, then implement an HTTP adapter after the backend team confirms the API and authentication contract.

Service interface first

```
export interface DeviceService {
  listDevices(): Promise<Device[]>;
  getDevice(deviceId: string): Promise<Device>;
}

export interface AlertService {
  listAlerts(query: AlertQuery): Promise<AlertPage>;
  acknowledgeAlert(alertId: string): Promise<AlertRecord>;
}
```

The interfaces describe application needs. They do not claim that the current backend implements any endpoint.

Offline queue rules

Rule	Implementation expectation
Write before send	Persist valid telemetry with an event ID before network upload.
Idempotency	Send the same event ID on retry so backend duplication can be prevented.
Bound storage	Set a maximum queue size/age approved by the project; show when records cannot be retained.
Retry responsibly	Use exponential backoff and network-aware retry; do not keep the radio/CPU busy.
Confirm receipt	Remove or mark delivered only after the backend confirms it.
Protect privacy	Keep only data necessary for relay; encrypt/secure local credentials and limit diagnostic logs.

API boundary validation

Validate every incoming response before display. A malformed response becomes a recoverable error state, not a screen filled with invented defaults. Handle 401 sign-in expiration, 403 forbidden, 404 unavailable record, 429 retry guidance, and 5xx/network failure separately. Server-side authorization must control access to patient/device/session IDs; hiding an app tab is not access control.

7 Authentication privacy and security

Security begins before cloud integration: do not place patient data, API tokens, Wi-Fi credentials, BLE secrets, or backend administrator credentials in source code, screenshots, Wokwi projects, or demo fixtures.

Mobile security baseline

Area	Required approach
Token storage	Use platform-backed secure storage such as Expo SecureStore for small secrets; do not use AsyncStorage for access tokens.
Transport	Use HTTPS with certificate/hostname verification. Do not accept insecure TLS to “fix” development errors.
Authorization	The backend verifies identity and scope on every protected operation. Mobile role filtering is UX only.
Logs	Do not log patient identifiers, tokens, payloads, or device credentials in production diagnostics.
Local data	Store the minimum data required; design expiry, sign-out clearing, and consent/retention rules with the project owner.
Updates	Pin dependency versions, review permission changes, and keep secrets out of build configuration committed to Git.

SecureStore notes

Expo SecureStore uses platform secure storage for small key-value secrets. It supports options that can require device authentication, but those prompts can fail or be cancelled and must be handled as part of the normal sign-in flow. Test on a real phone before relying on biometric protection.

Privacy and consent

- Collect only data required for the agreed prototype purpose.
- Use pseudonymous IDs in device queues and development logs where possible.
- Put privacy explanation, consent, data retention/deletion, export, and account-removal decisions in the approved project policy rather than inventing claims in the app.
- Do not capture background Bluetooth data, notifications, or location unless the product requirement, permissions, consent, and platform behavior are understood.

8 Testing strategy

A reliable mobile app needs several kinds of testing. Passing unit tests does not prove BLE behavior, server authorization, or physical wearable operation.

Layer	Test it with	What it proves
Pure rules	Unit tests	Status mapping, packet decoding, time/freshness, queue ordering, data conversion.
UI components	Component tests	Loading, error, empty, stale, accessibility labels, user interaction.
Navigation	Device/emulator flow	Screen transitions and unauthorized/expired-session states.
BLE	Physical Android phone + wearable	Permissions, scan/connect, notifications, disconnect/recovery, packet timing.
Backend	Staging contract tests	Authentication, authorization, schema validation, retry/idempotency.
End-to-end	Controlled bench session	Edge -> mobile -> backend flow, with timestamps and queues visible.

Test scenarios before calling the app ready

- No Bluetooth permission, Bluetooth switched off, no devices found, and user cancels pairing.
- Wrong device, missing characteristic, malformed/old/duplicate packet, and packet sequence gap.
- Wearable disconnects while a session is open, then recovers; no frozen “connected” UI.
- No network, backend returns 401/403/404/429/5xx, and queue reaches configured limit.
- Sign-out clears session-sensitive local state; token expiry returns to a safe sign-in flow.
- Screen reader names, text scaling, contrast, offline labels, and charts/table alternatives.
- Demo data visibly says Demo; hardware data visibly shows source, last successful time, units, and quality.

Useful acceptance evidence

For each milestone, save the app version, Android device model/OS version, screenshots with no real patient data, test steps, expected and actual result, and known limitation. Record an untested item as pending; do not call it passed.

9 A practical 8 phase build plan

Phase	Build	Exit gate
1	Project, TypeScript, navigation, design tokens	Android app opens; accessibility baseline checked.
2	Demo auth shell and role-aware navigation	No protected content shown without demo role context.
3	Device/sensor status with demo adapter	Unknown, stale, disconnected, and demo states distinct.
4	Session and alert views	Units, timestamps, text alternatives, error/empty/loading states.
5	BLE mock adapter and protocol document	UI behaves correctly with controlled synthetic packets.
6	Physical BLE adapter on Android	Scan, valid packet, disconnect, recovery, and rejection tests pass.
7	Confirmed backend/auth/queue	Schema validation, authorization, retries, idempotency, and token storage reviewed.
8	Hardening and pilot preparation	Regression, accessibility, privacy, security, and bench E2E evidence complete.

What not to build yet

- Firmware updates, clinical recommendations, automated limit changes, or emergency claims.
- A BLE protocol based only on guessing UUIDs or parsing raw packets without a versioned contract.
- Real patient accounts/data until backend, consent, privacy, and authorization decisions are approved.
- A cloud API based only on planning document interface names.

First implementation task

Create the Android-first React Native TypeScript app, then build one **Demo Device Status** screen using the proposed connection and measurement-quality states. Use synthetic values labelled Demo. This teaches project structure, navigation, accessible UI, state modelling, and testable service adapters before hardware and backend work begin.

10 References and decisions still needed

Use the current documentation below when implementing. Library and platform details change, so check the linked official documentation before installing packages or configuring release builds.

Primary references

- React Native environment setup: https://reactnative.dev/docs/set-up-your-environment
- React Native TypeScript: https://reactnative.dev/docs/typescript
- Expo development builds: https://docs.expo.dev/develop/development-builds/use-development-builds/
- Expo SecureStore: https://docs.expo.dev/versions/latest/sdk/securestore/
- React Navigation: https://reactnavigation.org/docs/getting-started/
- OrthoSense proposed models and API boundary: D:\3yp\planning_packages\docs\DATA_MODEL_AND_API_CONTRACTS.md
- OrthoSense route and role planning: D:\3yp\planning_packages\docs\ROUTES_AND_PERMISSIONS.md
- OrthoSense unresolved integration decisions: D:\3yp\planning_packages\docs\OPEN_QUESTIONS.md

Resolve these before real integration

Topic	Decision required
Backend	Actual base URLs, API schema, auth/session method, permissions, pagination, error format, and real-time transport.
BLE	Device identity, GATT UUIDs, packet encoding/versioning, pairing/authentication, command acknowledgements, reconnect policy.
Measurements	Definitions, units, calibration status, stale thresholds, timestamps, and how model outputs are labelled.
Privacy	Consent, retention, deletion/export, notification policy, and institutional requirements.
Product scope	Whether the first app is patient gateway, clinician companion, or both; which role ships first.

Current limitation. *This guide is grounded in the planning documents and the existing ESP32-S3 prototype work. It does not assert that a production backend, BLE GATT service, mobile authentication provider, or clinical validation already exists.*